

Event-Centric Memory with Query-Aware Graph Augmentation for Long-Term Conversational Agents

Yichen Liu, Chunfeng Yuan, Haowei Liu, Wenjuan Li, Zefeng Lin, Bing Li, Xu Chen, and Weiming Hu

Abstract—For persistent and personalized conversational agents, memory systems can enable them to remember, update, and reason over long histories by storing past interactions and retrieving relevant information. Existing memory systems typically follow two paradigms: flat-structured memory and graph-based memory. The former is lightweight but leaves event relations and state updates implicit, while the latter explicitly models memory structure but incurs additional construction cost and introduces irrelevant relations over long histories. To address these limitations, we propose QGMEM, a novel memory construction and activation framework motivated by human memory, in which experience is organized into events and query-relevant events are modeled by graph as working memory. QGMEM converts long dialogue histories into event-indexed atomic memory units that preserve individual experiences and consolidates related units into dynamic memory traces that retain state trajectories and current states. When a query arrives, hybrid memory retrieval gathers complementary candidate memories, and query-aware reranking activates the most relevant units as a compact working memory. To expose relational dependencies in the working memory and support conflict-aware reasoning, QGMEM organizes the working memory as a local graph, which is then encoded as a graph token and provided to the LLM together with the textual working memory to improve evidence utilization during answer generation. Experiments across six benchmarks validate the framework and show consistent gains in retrieval, multi-hop evidence composition, conflict resolution, and ultra-long dialogue reasoning with compact contexts and moderate inference cost.

Index Terms—Long-term memory, conversational agents, graph-based memory.

I. INTRODUCTION

Maintaining continuity across long-term interaction between humans and agents remains a pivotal milestone towards personalized and autonomous artificial intelligence. Personal assistants, companion dialogue systems, and task-oriented agents are expected to remember user preferences, interpersonal relations, schedules, and later updates across many turns

or sessions. Since such information cannot always fit into the model context [1], an agent needs memory mechanisms that preserve past interactions, track evolving states, identify relevant memory, and organize their relations for answer generation. Therefore, recent works [2]–[4] introduce long-term memory systems that store historical interactions as persistent external memory and retrieve relevant memory units when answering user queries.

Existing memory systems mainly follow two organizational paradigms, as illustrated in Figure 1(a) and (b). **Flat-structured memory** stores historical information as turns, sessions or summaries [3]–[5]. This paradigm is lightweight and easy to maintain, but relations among events and state changes are usually left implicit. **Graph-based memory** explicitly represents entities, events, and states as graph structures [6]–[8]. Such structures support relational reasoning, but constructing and retrieving graph structures over long histories introduces additional cost and even irrelevant relations. Consequently, existing long-term conversational memory systems struggle to simultaneously achieve lightweight persistent storage and explicit relational organization.

These limitations motivate a memory architecture that combines persistent storage with query-time relational organization: long-term memory preserves events and their evolving states, whereas working memory constructs a relational structure over query-relevant events. This view is consistent with classic research in cognitive psychology on event cognition, which suggests that people represent continuous experience as a sequence of events, update the current event in working memory, and retrieve relevant past events from long-term memory [9]–[11]. Motivated by this perspective, we propose Query-aware Graph-augmented Memory (QGMEM), which combines event-centric memory storage with query-time relational organization, as illustrated in Figure 1(c). QGMEM first converts dialogue histories into event-indexed atomic memory units with traceable source content, each representing an independently interpretable event or state and indexed by explicit event attributes, including entities, locations, and factual time. Related units are then consolidated into dynamic memory traces that preserve state trajectories and current states, thereby making state updates explicit and distinguishing current facts from superseded ones. To improve the retrieval quality of answer-bearing memories, hybrid memory retrieval integrates lexical, semantic, and event-attribute relevance using reciprocal rank fusion (RRF) [12]. Query-aware reranking removes irrelevant and superseded candidates, yielding a compact working memory concentrated on the current query. To model relational dependencies within the activated working memory, QGMEM constructs a local memory

- Y. Liu, C. Yuan, W. Li, Z. Lin, B. Li, and W. Hu are with the State Key Laboratory of Multimodal Artificial Intelligence Systems, Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China.
- Y. Liu, C. Yuan, B. Li, and W. Hu are also with the School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing 100049, China.
- C. Yuan, W. Li, Z. Lin, B. Li, and W. Hu are also with the Beijing Key Laboratory of Super Intelligent Security of Multi-Modal Information, Beijing, China.
- H. Liu is with Alibaba Group, China.
- B. Li is also with PeopleAI Inc., Beijing 100190, China.
- X. Chen is with the Gaoling School of Artificial Intelligence, Renmin University of China, Beijing 100872, China.
- W. Hu is also with the School of Information Science and Technology, ShanghaiTech University, Shanghai 201210, China.
- Corresponding author: Chunfeng Yuan (e-mail: cfyuan@nlpr.ia.ac.cn).

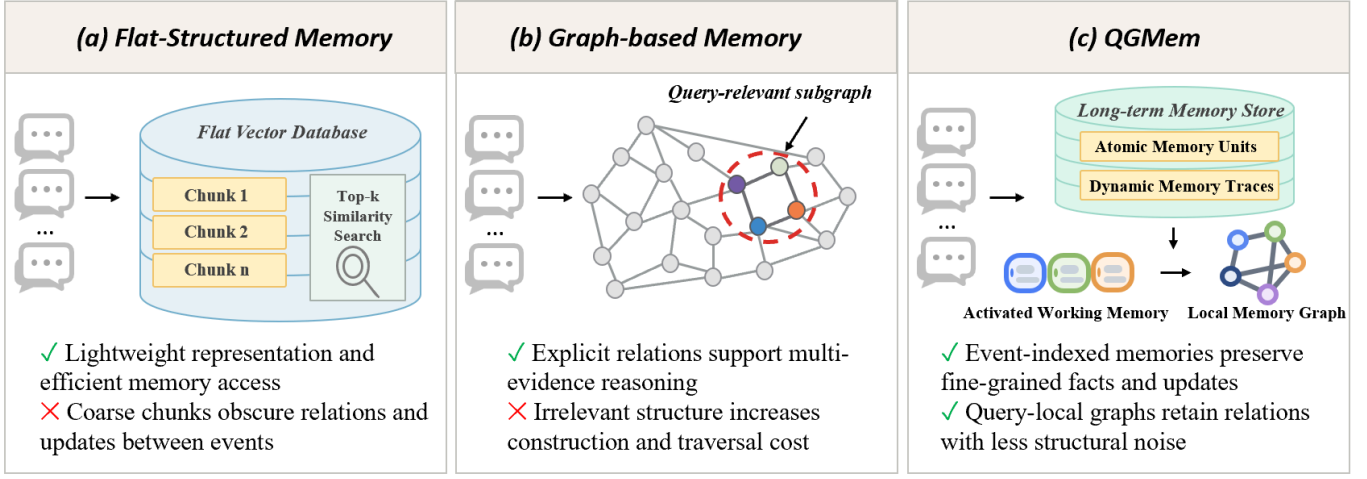


Fig. 1. Comparison of flat-structured memory, graph-based memory, and QGMEM.

graph from the facts of the selected memory units. Then, a relation-aware graph encoder is employed to encode the local memory graph, and the resulting graph-level representation is projected into the LLM embedding space. The graph token is supplied to the LLM together with the textual working memory, enabling multi-hop evidence composition based on both memory content and relational information.

The resulting framework is designed to reconcile three objectives in long-term conversational memory: preserving verifiable historical evidence, exposing query-relevant relations, and maintaining a compact generation context. Figure 2 provides an empirical view of this balance by comparing QGMEM with retrieval-augmented generation (RAG) and representative flat-structured and graph-based memory systems in terms of retrieval quality, answer quality, and context efficiency. Flat-structured systems maintain compact contexts but exhibit lower retrieval or answer quality, whereas graph-based systems do not simultaneously attain high retrieval and answer quality. In contrast, QGMEM achieves an $nDCG@10$ of 82.50% and an LLM-judged score of 82.77% with 598.19 retrieved-context tokens. These results show that QGMEM improves the prioritization and utilization of answer-supporting memories while maintaining a compact context for generation.

We conduct main experiments on six long-term memory benchmarks. LoCoMo [13] and LongMemEval-S [14] evaluate accurate retrieval and multi-hop evidence composition, while Fact-SH and Fact-MH from MemoryAgentBench [15] evaluate conflict resolution. BEAM-100K and BEAM-500K [16] assess scalability over ultra-long dialogue histories. We further extend the BEAM evaluation to 1M and 10M tokens to examine performance under extreme context lengths. Overall, QGMEM consistently outperforms state-of-the-art systems. Further evaluations show that these gains remain consistent across two open-weight backbones, Qwen3-8B and Llama3.1-8B-Instruct, and one closed-source backbone, GPT-4o-mini [17]–[19]. In-depth ablation experiments confirm the effectiveness of the key components in QGMEM. Additional efficiency, scalability, hyperparameter sensitivity, and case analyses characterize the computational cost, robustness, and retrieval performance of QGMEM from complementary perspectives.

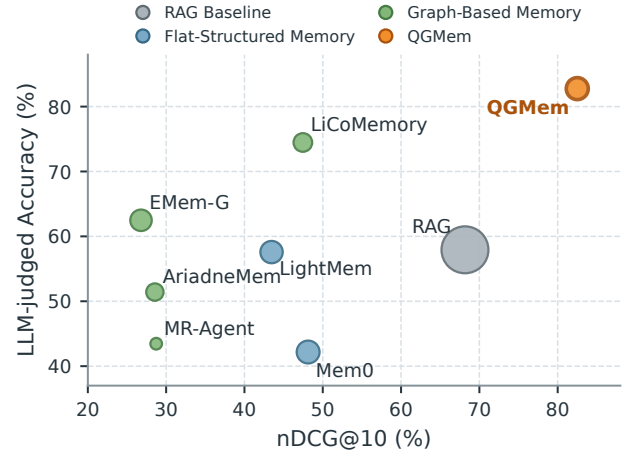


Fig. 2. Comparison of retrieval quality, LLM-judged answer score, and retrieved-context size on LoCoMo with Qwen3-8B. Colors distinguish the retrieval-augmented baseline, flat-structured memory, graph-based memory, and QGMEM. The horizontal axis denotes normalized discounted cumulative gain at rank 10 ($nDCG@10$), the vertical axis denotes the LLM-judged score, and bubble area denotes the average number of retrieved-context tokens. Higher axis values and smaller bubble areas indicate better overall performance.

The main contributions of this work are summarized as follows:

- We introduce an event-centric memory framework for persistent conversations that integrates fine-grained event storage, evolving-state consolidation, query-aware activation, and relational evidence organization within a unified memory lifecycle.
- We represent conversational experience using traceable atomic memory units and dynamic memory traces, preserving both historical states and currently valid information as the dialogue evolves.
- We organize query-relevant memories into a compact working memory and encode their local relational topology jointly with source-grounded textual evidence, supporting evidence composition without maintaining a graph over the complete dialogue history.
- We evaluate the framework across six benchmarks and multiple LLM backbones, examining answer quality,

retrieval effectiveness, conflict-sensitive reasoning, long-context scalability, and computational cost.

II. RELATED WORK

A. Persistent Conversational Memory Representation

A central design choice in conversational memory is how interaction histories are represented and organized for persistent storage [2], [8], [20], [21]. Flat-structured memory retains dialogue turns or transforms them into chunks, summaries, and facts. MemGPT manages prior interactions through external memory [4], while LongMem maintains a retrievable long-term memory cache [22]. MemoryBank and FACT organize summarized or extracted information [3], [5], whereas Mem0 and LightMem consolidate salient records to improve compactness and access efficiency [23], [24]. These representations are easy to maintain and can be supplied directly to an LLM, but event relations and state changes remain implicit.

Graph-based conversational memory, by contrast, materializes explicit associations at different granularities. A-MEM organizes memory notes into an evolving network through dynamically generated links and updates [25]. Associa constructs an event-centric memory graph and combines subgraph extraction with iterative deliberative recall [26]. Zep maintains a temporal knowledge graph that records the validity and history of relations [27]; SG-Mem connects sentences within chunked dialogue units [6]; and EMem-G organizes sessions, event-like discourse units, and their arguments in a heterogeneous graph [7]. These approaches differ in graph granularity and edge semantics, while sharing the use of an explicit graph as a persistent memory index.

Recent systems further enrich persistent graphs with hierarchical, temporal, and provenance-aware organization. LiCoMemory and GAM connect detailed events with higher-level semantic structures [28], [29], while HyperMem uses hyperedges to capture higher-order associations among topics, episodes, and facts [30]. APEX-MEM adopts an append-only temporal property graph to preserve evolving facts [31], and MemORAI couples a provenance-enriched multi-relational graph with query-adaptive edge weighting [32]. Complementary structured-memory approaches such as StructMem preserve event-level bindings and periodically consolidate cross-event connections without imposing a persistent entity–relation graph [33]. A controlled analysis of long-term dialogue memory systems further shows that graph effectiveness depends jointly on memory-unit design, graph maintenance, seed activation, and graph expansion, making the surrounding retrieval pipeline important for interpreting gains from graph structure [34].

Persistent graphs expose relational structure throughout the memory lifecycle, but relation extraction, alignment, and maintenance operate over an expanding history. QGMEM places graph materialization after query-aware activation. Its persistent substrate consists of source-grounded atomic units and lightweight dynamic traces; a relational graph is instantiated only over the activated working memory. Dynamic traces preserve longitudinal state evolution, while the query-local graph organizes dependencies among the selected facts. This separation limits the scope of graph construction without discarding historical evidence or its provenance.

B. Query-Time Memory Access

Effective use of persistent conversational memory depends on retrieving information that is relevant and sufficient for the current query without introducing extraneous context. Flat-structured memory systems generally rank independent records by lexical or semantic relevance. Lexical retrieval, commonly implemented with BM25 [35], preserves exact names, dates, and other discriminative expressions, but misses records expressed in substantially different terms. Dense retrieval captures semantic correspondence beyond exact terms [36]–[38] and is used by systems such as LongMem and Mem0 [22], [23]. However, descriptions of similar events may receive high semantic relevance despite referring to different entities, times, or states.

Retrieval over a persistent graph commonly begins with semantically matched seed nodes and then expands through stored edges [34]. Associa extracts evidence-rich subgraphs through prize-collecting Steiner tree optimization [26]; Ariadne-Mem discovers bridge paths between distributed evidence [39]; HyperMem performs coarse-to-fine retrieval over its topic, episode, and fact hierarchy [30]; and GAM follows cross-level links from relevant topics to archived event graphs [29]. MemORAI applies query-conditioned edge weighting during graph propagation [32], while APEX-MEM uses a retrieval agent to resolve conflicting and evolving facts [31]. These strategies exploit stored topology to broaden evidence coverage, while their retrieval behavior remains coupled to the construction and maintenance of the persistent graph.

To address these limitations, QGMEM applies hybrid retrieval over event-indexed atomic memory units to improve the coverage of answer-supporting information while reducing the retrieval of events whose attributes do not match the query. Query-aware reranking then filters irrelevant and superseded candidates to form a compact working memory. A local graph constructed over this activated memory preserves explicit relational organization for evidence composition while avoiding the maintenance cost and query-irrelevant relations associated with a persistent global graph.

C. Memory-Augmented Language Generation

Beyond selecting relevant records, memory-augmented generation determines how retrieved information is incorporated into the LLM. Most systems provide retrieved memories to the LLM as text: flat records are appended directly to the prompt, while graph-based memory is serialized as triples, paths, subgraphs, or summaries [27], [39]–[41]. Although broadly compatible with language models, serialization linearizes graph topology, requiring the LLM to reconstruct structural dependencies from tokens and increasing context consumption.

Recent studies on integrating graph structures with LLMs have shown that learned graph representations can preserve topology and provide relational signals directly during generation, without requiring the LLM to reconstruct these dependencies solely from serialized text [42], [43]. G-Retriever combines an encoded subgraph with its textual representation for graph question answering [44], while RAS and Relink construct query-specific evidence graphs for knowledge-intensive

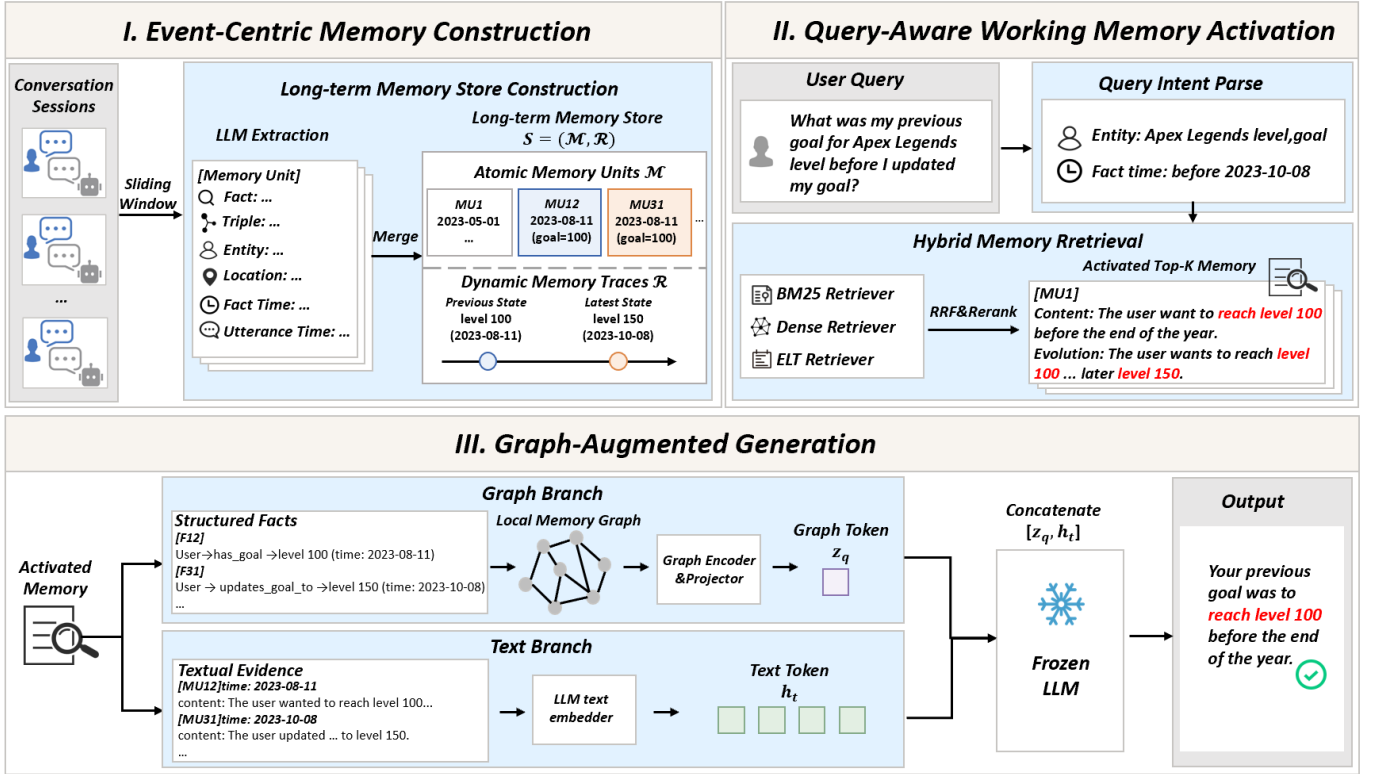


Fig. 3. Overall framework of QGMEM.

generation and GraphRAG [45], [46]. QGMEM builds on the general principle of query-conditioned graph augmentation but addresses persistent conversational memory, where facts may be introduced, revised, supplemented, or superseded over time. It therefore couples query-local relational organization with event-indexed atomic storage and dynamic memory traces, allowing generation to use selected evidence together with its retained state history.

III. METHOD

The overall framework of QGMEM is illustrated in Figure 3 and comprises three modules. First, **event-centric memory construction** converts the dialogue history into event-indexed atomic memory units and consolidates related units into dynamic memory traces, preserving verifiable evidence and evolving states. Second, **query-aware working memory activation** retrieves and reranks memories for the query q , forming a compact working memory that concentrates complete, current, and answer-relevant information. Finally, **graph-augmented generation** organizes the activated memories into a local graph and jointly provides its graph token and textual working memory to the LLM, enabling answer generation from both memory content and structural relations. In the following, we will introduce these three modules in detail, and analyze the generalization and efficiency of our proposed framework.

A. Event-Centric Memory Construction

QGMEM represents a dialogue history as an event-indexed atomic memory set $\mathcal{M}$ and a dynamic trace set $\mathcal{R}$. Each atomic unit represents a fine-grained event or state and retains

its source dialogue, structured fact, and event and temporal attributes for traceable retrieval. Dynamic traces connect related units into state trajectories that distinguish current information from superseded records. Together, they form the long-term memory store $\mathcal{S} = (\mathcal{M}, \mathcal{R})$, supporting precise memory access and conflict-aware answering over evolving histories.

1) *Atomic Memory Unit Construction*: To construct atomic memories, QGMEM first scans each dialogue session with N_W overlapping sliding windows $\{W_i\}_{i=1}^{N_W}$. Each window W_i provides a local dialogue context that contains newly processed turns together with limited preceding context for resolving pronouns and local references. Given W_i , QGMEM decomposes the dialogue into independently retrievable memory units with traceable source content, each representing a single interpretable event or state. When one extracted candidate contains multiple independently retrievable relations, they are materialized as separate atomic units so that each finalized unit is anchored by one complete structured fact. Let K_i be the number of units extracted from W_i , and let $m_{i,j}$ denote its j -th unit. The complete memory set is $\mathcal{M} = \{m_{i,j} \mid 1 \leq i \leq N_W, 1 \leq j \leq K_i\}$. For a generic unit $m \in \mathcal{M}$, c_m denotes its context-independent summary, d_m the corresponding source dialogue span, $\mathcal{E}_m$ and $\mathcal{P}_m$ its entity and location sets, and f_m its structured fact. The factual and utterance times of the unit are denoted by T_m^f and T_m^u . The unit is represented as

$$m = (c_m, d_m, \mathcal{E}_m, \mathcal{P}_m, T_m^f, T_m^u, f_m). \quad (1)$$

Together, these fields preserve source traceability and expose explicit event and temporal attributes for trace consolidation and retrieval.

This representation is designed to make each memory unit independently retrievable while preserving the event and temporal context required for subsequent reasoning. It has two advantages: (1) **Multi-perspective event matching**. Explicit entity, location, and factual-time attributes characterize an event or state from complementary perspectives, helping distinguish answer-bearing memories from topically related events and improving their retrieval priority. (2) **Dual-time representation**. Separating factual time T_m^f from utterance time T_m^u makes temporal information directly comparable during retrieval. For example, if a user said on January 20, “I went to the hospital yesterday,” then T_m^u is January 20, while T_m^f is normalized to January 19. Resolving this distinction during memory construction supports temporal grounding and selection of the valid memory under time-sensitive or conflicting records.

2) *Dynamic Memory Trace Consolidation*: Atomic memory units preserve fine-grained evidence, but long-term conversations often contain state changes that cannot be captured by isolated units alone. For example, a user may first set a goal, later revise it, and finally ask about the previous goal. To make such update trajectories explicit, QGMEM consolidates related memory units into dynamic memory traces. Write the structured fact of unit m as $f_m = (a_m, p_m, o_m)$, where a_m , p_m , and o_m are its subject, predicate, and object, respectively. With the normalized text encoder $E(\cdot)$, we first compute three pairwise similarities and then combine them into a relation score:

$$\begin{aligned} s_f^{ij} &= \cos(E(f_{m_i}), E(f_{m_j})), \\ s_a^{ij} &= \cos(E(a_{m_i}), E(a_{m_j})), \\ s_p^{ij} &= \cos(E(p_{m_i}), E(p_{m_j})), \\ S_{\text{rel}}(m_i, m_j) &= \min\{s_f^{ij}, s_a^{ij}, s_p^{ij}\}. \end{aligned} \quad (2)$$

Here, s_a^{ij} and s_p^{ij} ensure that two facts describe the same state holder and relation, while s_f^{ij} rejects otherwise unrelated assertions without requiring identical objects. We take their minimum because a false merge can corrupt an entire trajectory, whereas unmerged units remain independently retrievable. For each unit, we test its 20 nearest fact-level neighbors. A set of memory units forms a relation cluster only if the relation score between every pair of their structured facts is no smaller than τ_r . We set $\tau_r = 0.80$ based on the sensitivity analysis in Figure 7.

For the h -th cluster, ψ_h denotes its representative subject-predicate relation. An LLM-based consolidator orders the facts chronologically, merges repeated or semantically equivalent states, and retains distinct state changes. It treats a later incompatible value as an *update*, a compatible refinement as a *supplement*, and mutually compatible values as *coexisting states*. The resulting trace is $r_h = (\psi_h, H_h, s_h)$, where H_h is the ordered state trajectory and s_h is its latest supported state:

$$r_h = (\psi_h, H_h, s_h). \quad (3)$$

If consolidation produces N_R traces, the resulting trace set is

$$\mathcal{R} = \{r_h\}_{h=1}^{N_R}. \quad (4)$$

At query time, each retrieved atomic unit remains linked to its dynamic trace. The trace supplies the state trajectory to

the reranker and generator, while the latest member provides a current-state representative when several retrieved units describe different versions of the same relation. This separation preserves source-level granularity for verification and retrieval, while linking distant updates into a coherent trajectory for conflict resolution and multi-hop evidence composition.

B. Query-Aware Working Memory Activation

QGMEM activates working memory by selecting a compact set of query-relevant memory units from the long-term memory store $\mathcal{S} = (\mathcal{M}, \mathcal{R})$. The procedure follows two successive steps: hybrid memory retrieval and query-aware reranking. Hybrid memory retrieval identifies complementary candidates through lexical, semantic, and event-attribute retrieval. Lexical retrieval preserves exact query cues, semantic retrieval identifies related expressions, and event-attribute retrieval distinguishes memories with similar content but different entities, locations, or temporal conditions. RRF [12] combines the three ranked lists according to rank positions, producing a unified candidate ordering without cross-channel score calibration. Query-aware reranking subsequently evaluates the fused candidates and selects the final working memory under a fixed budget, prioritizing memories that are relevant and consistent with the current state. Algorithm 1 summarizes the complete procedure.

1) *Hybrid Memory Retrieval*: Given a query q issued at time t_q , QGMEM retrieves candidate memory units $m \in \mathcal{M}$ through three complementary channels. Lexical retrieval, implemented with BM25 [35], identifies units containing exact query cues such as names, dates, and locations, whereas semantic retrieval identifies related units expressed through aliases or paraphrases using dense embeddings. For event-attribute retrieval, the parser $\phi(q, t_q)$ extracts the entity, location, and temporal constraint sets $\mathcal{C}_q^{\text{ent}}$, $\mathcal{C}_q^{\text{loc}}$, and $\mathcal{C}_q^{\text{time}}$. SemMatch evaluates the compatibility of entity and location constraints with $\mathcal{E}_m$ and $\mathcal{P}_m$, while TimeMatch evaluates the temporal constraint against (T_m^f, T_m^u) . Writing $\mathcal{C}_q^x$ for the corresponding constraint set, the available scores are averaged over nonempty constraints to obtain S_{ELT} .

Each channel retains its n highest-scoring units and produces a ranked list $\mathcal{L}_b$, $\mathcal{L}_d$, or $\mathcal{L}_e$. We use $\text{TopK}_{x \in \mathcal{X}}^b s(x)$ to denote the b highest-scoring elements of $\mathcal{X}$ under score s . RRF [12] assigns the fused score $S_{\text{RRF}}(m, q)$ according to $\text{rank}_{\mathcal{L}}(m)$, the position of m in list $\mathcal{L}$. Its rank-offset constant κ is fixed to 60. The n highest-scoring units after fusion form the reranking candidate pool $\mathcal{C}_0$. This fusion preserves complementary retrieval coverage without requiring score calibration, particularly when long histories contain many superficially similar memories.

2) *Query-Aware Reranking*: Candidates in $\mathcal{C}_0$ still differ in content completeness and answer relevance because first-stage scores consider each signal separately. QGMEM therefore constructs a reranking document for every candidate containing its context-independent fact summary, source dialogue, entities, locations, factual time, utterance time, and the state trajectory from its associated dynamic memory trace when available. $\text{Reranker}(q, m)$, implemented with Qwen3-Reranker-8B, jointly scores the query and this structured document. Under

the working-memory budget k , the highest-scoring candidates form the activated working memory $\mathcal{M}_q$. Its ordering is used by both graph construction and answer generation. This two-stage activation preserves candidate-stage coverage while concentrating complete, current, and answer-relevant memories at the highest ranks. The bounded working context retains the evidence needed for generation with controlled input size, supporting retrieval quality, context efficiency, and stable access as the memory store grows.

C. Graph-Augmented Generation

The graph token encodes relational topology, while temporal order and state validity remain in the dynamic traces and serialized working memory. These complementary inputs support source-grounded, relation-aware generation without constructing a graph over the full history.

1) *Local Memory Graph Construction*: Each structured fact $f_m = (u, \rho, v)$ in the activated working memory $\mathcal{M}_q$ is represented by an entity node for the subject u , a value node for the object v , and a directed edge labeled by the predicate ρ . The mappings $\nu_{\text{ent}}(\cdot)$ and $\nu_{\text{val}}(\cdot)$ normalize entity and value labels, respectively, such that their repeated occurrences share the same node. The induced node and directed-edge sets are

$$\begin{aligned} \mathcal{V}_q &= \bigcup_{m \in \mathcal{M}_q} \{\nu_{\text{ent}}(u_m), \nu_{\text{val}}(v_m)\}, \\ \mathcal{A}_q &= \left\{ (\nu_{\text{ent}}(u_m), \rho_m, \nu_{\text{val}}(v_m)) \mid f_m = (u_m, \rho_m, v_m) \right\}. \end{aligned} \quad (5)$$

The resulting local memory graph is $\mathcal{G}_q = (\mathcal{V}_q, \mathcal{A}_q)$. It encodes the relational topology of the activated facts, supporting relation-aware evidence composition. Time-aware retrieval and dynamic traces in the text branch preserve temporal order and state validity, complementing the graph topology during generation. Because $\mathcal{G}_q$ is induced by the fixed-budget working memory $\mathcal{M}_q$, its size remains bounded as the dialogue history grows.

2) *Graph Encoding*: QGMem initializes each node i and edge predicate ρ_e with the embeddings $\mathbf{x}_i$ and $\mathbf{r}_e$ produced by the fixed sentence encoder f_{emb} , which is shared with semantic retrieval.

To incorporate predicate information during message passing, the graph encoder applies L relation-aware TransformerConv layers [47] with H attention heads. For the incoming edge set $\mathcal{I}_q(i) = \{e = (j, \rho_e, i) \in \mathcal{A}_q\}$, the node states are initialized by $\mathbf{h}_i^{(0)} = \mathbf{x}_i$. At layer ℓ and head h , the relation-aware update is

$$\begin{aligned} \eta_{ie}^{(\ell, h)} &= \frac{(\mathbf{W}_Q^{(\ell, h)} \mathbf{h}_i^{(\ell)})^\top (\mathbf{W}_K^{(\ell, h)} \mathbf{h}_j^{(\ell)} + \mathbf{W}_E^{(\ell, h)} \mathbf{r}_e)}{\sqrt{d_h}}, \\ \alpha_{ie}^{(\ell, h)} &= \frac{\exp(\eta_{ie}^{(\ell, h)})}{\sum_{e' \in \mathcal{I}_q(i)} \exp(\eta_{ie'}^{(\ell, h)})}, \\ \mathbf{m}_i^{(\ell, h)} &= \sum_{e=(j, \rho_e, i) \in \mathcal{I}_q(i)} \alpha_{ie}^{(\ell, h)} (\mathbf{W}_V^{(\ell, h)} \mathbf{h}_j^{(\ell)} + \mathbf{W}_E^{(\ell, h)} \mathbf{r}_e), \\ \mathbf{h}_i^{(\ell+1)} &= \Gamma_\ell \left(\mathbf{W}_0^{(\ell)} \mathbf{h}_i^{(\ell)} + \left\| \mathbf{m}_i^{(\ell, h)} \right\|_{h=1}^H \right). \end{aligned} \quad (6)$$

Here, d_h is the dimension of each attention head and all $\mathbf{W}$ terms and Γ_ℓ are learned transformations. The attention weights jointly depend on neighboring node states and

predicate embeddings. Their aggregation therefore propagates both memory content and typed relations into the updated node representations. After L layers, global mean pooling summarizes the node representations as $\mathbf{g}_q$, and a two-layer nonlinear projector maps it to the memory graph token $\mathbf{z}_q$ in the LLM embedding space:

Algorithm 1 Query-Aware Working Memory Activation

Require: Memory set $\mathcal{M}$; query q at t_q ; budgets n, k ; RRF constant κ

Ensure: Activated working memory $\mathcal{M}_q$

```

1:  $(C_q^{\text{ent}}, C_q^{\text{loc}}, C_q^{\text{time}}) \leftarrow \phi(q, t_q)$  ▷ query event constraints
2:  $\mathcal{L}_b \leftarrow \text{TopK}_{m \in \mathcal{M}}^n \text{BM25}(q, m)$  ▷ lexical retrieval
3:  $\mathcal{L}_d \leftarrow \text{TopK}_{m \in \mathcal{M}}^n \text{Dense}(q, m)$  ▷ semantic retrieval
4:  $\mathcal{L}_e \leftarrow \emptyset$ 
5: if  $C_q^{\text{ent}} \neq \emptyset \vee C_q^{\text{loc}} \neq \emptyset \vee C_q^{\text{time}} \neq \emptyset$  then
6:   for  $m \in \mathcal{M}$  do
7:     if  $C_q^{\text{ent}} \neq \emptyset$  then
8:        $s_{\text{ent}}(m, q) \leftarrow \text{SemMatch}(C_q^{\text{ent}}, \mathcal{E}_m)$ 
9:     end if
10:    if  $C_q^{\text{loc}} \neq \emptyset$  then
11:       $s_{\text{loc}}(m, q) \leftarrow \text{SemMatch}(C_q^{\text{loc}}, \mathcal{P}_m)$ 
12:    end if
13:    if  $C_q^{\text{time}} \neq \emptyset$  then
14:       $s_{\text{time}}(m, q) \leftarrow \text{TimeMatch}(C_q^{\text{time}}, (T_m^f, T_m^u))$ 
15:    end if
16:     $\text{SELT}(m, q) \leftarrow \text{mean}_{x \in \{\text{ent}, \text{loc}, \text{time}\}}: s_x(m, q)$ 
17:  end for
18:   $\mathcal{L}_e \leftarrow \text{TopK}_{m \in \mathcal{M}}^n \text{SELT}(m, q)$  ▷ event-attribute retrieval
19: end if
20:  $\mathcal{U} \leftarrow \mathcal{L}_b \cup \mathcal{L}_d \cup \mathcal{L}_e$ 
21: for  $m \in \mathcal{U}$  do
22:    $S_{\text{RRF}}(m, q) \leftarrow \sum_{\substack{\mathcal{L} \in \{\mathcal{L}_b, \mathcal{L}_d, \mathcal{L}_e\} \\ m \in \mathcal{L}}} (\kappa + \text{rank}_{\mathcal{L}}(m))^{-1}$ 
23: end for
24:  $\mathcal{C}_0 \leftarrow \text{TopK}_{m \in \mathcal{U}}^n S_{\text{RRF}}(m, q)$ 
25:  $\mathcal{M}_q \leftarrow \text{TopK}_{m \in \mathcal{C}_0}^k \text{Reranker}(q, m)$  ▷ working memory
26: return  $\mathcal{M}_q$ 

```

$$\begin{aligned} \mathbf{g}_q &= \frac{1}{|\mathcal{V}_q|} \sum_{i \in \mathcal{V}_q} \mathbf{h}_i^{(L)}, \\ \mathbf{z}_q &= \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{g}_q + \mathbf{b}_1) + \mathbf{b}_2. \end{aligned} \quad (7)$$

The resulting $\mathbf{z}_q$ is supplied to the LLM. The graph encoder and projector parameters are collected in ω , with their configuration reported in Section B of the Appendix.

3) *Graph-Text Fusion*: The LLM text embedder converts the serialized working memory C_q into the textual token sequence $\mathbf{h}_t$. As illustrated in Figure 3, graph-text fusion concatenates the graph token $\mathbf{z}_q$ with $\mathbf{h}_t$ and supplies $[\mathbf{z}_q, \mathbf{h}_t]$ to the frozen LLM together with the query q . For an answer sequence $\mathbf{a}$, the resulting autoregressive distribution is $p_{\theta, \omega}(a_j \mid \mathbf{a}_{<j}, q, [\mathbf{z}_q, \mathbf{h}_t])$, where θ and ω are the frozen LLM parameters and trainable graph-module parameters, respectively. This fusion preserves readable, source-grounded memory content while supplying graph-structural signals for relation-aware generation.

4) *Training Objective*: QGMEM first builds a persistent memory store through event-centric memory construction. Query-aware working memory activation and graph-augmented generation then transform query-relevant memories into textual and structural representations, which are supplied to an LLM for answer generation. These memory operations are decoupled from the LLM backbone model, allowing the framework to support different LLM backbone models through an appropriate generation interface. Accordingly, optimization is restricted to the parameters of the relation-aware graph encoder and the

two-layer projector, whereas both the sentence encoder and the backbone LLM remain fixed throughout training. Given N_D graph-grounded training instances, the s -th instance comprises a query $q^{(s)}$, serialized working memory $C_q^{(s)}$, local graph $\mathcal{G}_q^{(s)}$, and target sequence $\mathbf{a}^{(s)}$. The graph branch produces $\mathbf{z}_q^{(s)}$, while the text branch produces $\mathbf{h}_t^{(s)}$; their concatenation conditions the frozen LLM together with $q^{(s)}$. The objective is evaluated over the target tokens, whose total number is $T_D = \sum_{s=1}^{N_D} |\mathbf{a}^{(s)}|$. The teacher-forced objective is

$$\mathcal{J}(\omega) = -\frac{1}{T_D} \sum_{s=1}^{N_D} \sum_{j=1}^{|\mathbf{a}^{(s)}|} \log p_{\theta, \omega} \left(a_j^{(s)} \mid \mathbf{a}_{<j}^{(s)}, q^{(s)}, [\mathbf{z}_q^{(s)}, \mathbf{h}_t^{(s)}] \right). \quad (8)$$

We minimize $\mathcal{J}(\omega)$ in (8) with respect to ω . Consequently, answer supervision encourages the encoded graph representation to retain relational information that contributes to generation without altering the pretrained LLM. Detailed training settings are provided in Section B of the Appendix.

D. Backbone Adaptability and Efficiency Analysis

1) *Adaptability Across LLM Backbones*: QGMEM supports different backbone models, enabling the complete framework to operate with both open-weight and proprietary LLMs without reliance on a specific model. For open-weight backbones, the projector maps the encoded local graph to a graph token in the embedding space of the target LLM, which is provided jointly with the textual working memory. For proprietary backbones accessed through an API, the embedding space is inaccessible; accordingly, the activated working memory and local graph are serialized as text. This interface-level adaptation preserves the memory construction, activation, and graph organization procedures across backbones while accommodating their respective access constraints.

2) *Efficiency Analysis*: Long-term memory construction is performed offline once, and the resulting atomic units, dynamic traces, and retrieval indexes are reused across queries. Atomic memory construction processes the dialogue through fixed-size overlapping windows. Therefore, the number of extraction operations grows linearly with the number of dialogue turns. These windows can be processed independently, and their construction cost is amortized over all subsequent queries. For dynamic trace consolidation, each atomic unit examines only its 20 nearest fact-level neighbors, avoiding exhaustive pairwise comparison over the complete memory store.

At query time, retrieval and reranking activate at most k atomic memory units. Since each activated unit contributes one primary structured fact, the induced local graph contains at most k directed edges and $2k$ nodes before node deduplication. Local graph materialization therefore requires $O(k)$ time and space. With the best working-memory budget $k = 10$, both graph construction and graph encoding operate on a bounded structure whose size does not grow with the complete dialogue history. This design confines explicit relational organization to query-relevant evidence while reusing the lightweight persistent memory across queries.

IV. EXPERIMENTS

We evaluate QGMEM across complementary requirements for agent memory [48]: accurate retrieval, conflict resolution, ultra-long-history scalability, backbone adaptability, and efficiency. We further analyze retrieval quality, component contributions, hyperparameters, and a representative case.

A. Experimental Setup

1) *Benchmarks*: For **accurate retrieval**, LoCoMo [13] tests evidence retrieval and composition across multi-session dialogues, while LongMemEval-S [14] tests persistent recall, reasoning, and updates. For **conflict resolution**, FactConsolidation-SH and FactConsolidation-MH from MemoryAgentBench [15] cover single- and multi-hop selection among conflicting facts. For **ultra-long scalability**, BEAM [16] provides histories of 100K, 500K, 1M, and 10M tokens.

2) *Baselines*: We compare QGMEM with seven representative systems. The **retrieval-augmented baseline** is RAG; the **flat-structured memory baselines** are Mem0 [23] and LightMem [24]; and the **graph-based memory baselines** are EMem-G [7], LiCoMemory [28], MR-Agent [49], and AriadneMem [39]. We use the same questions and backbone model for all available systems on each benchmark. Further implementation details are provided in Section D of the Appendix.

3) *Evaluation Metrics*: We use two answer-quality metrics. (1) **LLM-judged Score**. A judge evaluates whether each answer satisfies the reference or task rubric [50]; we repeat judging 10 times and report mean and standard deviation. For BEAM, we follow its official protocol, scoring nine nugget-based abilities and Event Ordering with normalized Kendall's τ_b , then macro-averaging the ten ability scores. (2) **Answer Semantic Coverage**. AlignScore-large [51] measures coverage of reference information; uncertainty is estimated with 10,000 bootstrap resamples.

B. Quantitative Evaluation

1) *Accurate Retrieval*: Table I shows that QGMEM leads on LoCoMo and LongMemEval-S under both metrics. Its LLM-judged scores of 82.77% and 62.14% exceed the strongest baselines by 8.30 and 5.72 points; semantic coverage reaches 56.31% and 51.31%, gains of 12.77 and 2.78 points. Agreement across the two evaluators indicates that event-indexed storage and query-aware activation prioritize complete answer-bearing memories across multi-session evidence.

2) *Conflict Resolution*: On Fact-SH and Fact-MH, QGMEM attains LLM-judged scores of 78.10% and 12.50%, surpassing the strongest baselines by 2.18 and 6.00 points. Fact-MH nearly doubles the strongest baseline, with semantic coverage improving by 5.69 points to 13.34%; Fact-SH coverage reaches 80.58%. The larger multi-hop gain supports the division of labor between dynamic traces and time-aware activation for state selection and the local graph for evidence composition.

TABLE I
MAIN RESULTS WITH QWEN3-8B ACROSS SIX LONG-TERM MEMORY BENCHMARKS. BOLDFACE AND UNDERLINING INDICATE THE BEST AND SECOND-BEST PERFORMANCE RESPECTIVELY.

Method	Accurate Retrieval		Conflict Resolution		Ultra-Long Scalability	
	LoCoMo <i>N=1540</i>	LongMemEval-S <i>N=500</i>	Fact-SH <i>N=400</i>	Fact-MH <i>N=400</i>	BEAM-100K <i>N=400</i>	BEAM-500K <i>N=700</i>
(a) LLM-judged Score (%)						
<i>Retrieval-Augmented Baseline</i>						
RAG	57.93±0.06	34.74±0.10	37.82±0.24	3.93±0.17	29.04	28.00
<i>Flat-Structured Memory Baselines</i>						
Mem0	42.18±0.18	25.96±0.08	75.92±0.17	5.08±0.24	<u>31.61</u>	<u>33.50</u>
LightMem	57.58±0.23	<u>56.42±0.18</u>	62.25±0.17	<u>6.50±0.00</u>	27.79	29.75
<i>Graph-Based Memory Baselines</i>						
EMem-G	62.48±0.18	49.00±0.16	36.85±1.13	4.30±0.16	16.14	16.13
LiCoMemory	<u>74.47±0.10</u>	47.20±0.14	22.25±0.19	1.50±0.12	13.34	13.03
MR-Agent	43.46±0.15	23.20±0.12	20.50±0.21	3.00±0.15	12.89	16.85
AriadneMem	51.42±0.10	11.00±0.09	64.50±0.28	4.75±0.18	27.41	28.70
QGMEM (ours)	82.77±0.13	62.14±0.13	78.10±0.24	12.50±0.00	35.31	38.91
(b) Answer Semantic Coverage (%)						
<i>Retrieval-Augmented Baseline</i>						
RAG	<u>43.54±1.14</u>	29.85±1.84	38.71±2.39	5.00±0.95	7.32±1.00	8.54±0.82
<i>Flat-Structured Memory Baselines</i>						
Mem0	24.75±0.99	26.54±1.64	81.70±1.87	2.95±0.71	8.82±1.14	<u>11.50±0.95</u>
LightMem	35.45±1.13	<u>48.53±1.81</u>	61.00±2.27	<u>7.65±1.16</u>	<u>8.88±1.15</u>	9.50±0.86
<i>Graph-Based Memory Baselines</i>						
EMem-G	34.57±1.13	18.37±1.60	40.09±2.41	7.57±1.21	5.86±0.81	5.98±0.57
LiCoMemory	35.97±1.10	38.30±1.96	22.06±1.96	2.33±0.57	4.02±0.76	4.20±0.57
MR-Agent	20.44±0.91	21.03±1.70	20.39±1.88	3.79±0.82	3.98±0.66	5.61±0.59
AriadneMem	23.44±0.97	6.93±0.93	66.42±2.33	5.71±1.02	7.05±0.94	10.07±0.90
QGMEM (ours)	56.31±1.08	51.31±1.91	80.58±1.95	13.34±1.63	16.15±1.57	16.95±1.20

3) *Ultra-Long Dialogue History Scalability*: Across both scales, QGMEM achieves overall scores of 35.31%/38.91% on BEAM-100K/500K, exceeding Mem0 by 3.70/5.41 points, and semantic coverage of 16.15%/16.95%, with gains of 7.27/5.45 points. Its nine-ability Nugget Scores are 36.81%/40.26%, with Event Ordering τ_b of 21.77%/26.79%, showing that compact activation supports diverse memory abilities under long histories.

Figure 4 extends BEAM from 100K to 10M tokens; methods share questions and histories within each scale, while question sets differ across scales. QGMEM leads throughout, remains above 35% through 1M tokens, and scores 29.81% at 10M, exceeding LightMem and AriadneMem by 20.18 and 21.21 points. Event-indexed retrieval limits irrelevant-history interference, reranking selects a fixed-budget working memory, and the local graph grows with activated evidence rather than full history.

4) *Retrieval Quality Analysis*: Recall@10 measures gold-evidence coverage, Precision@10 the concentration of relevant units, Sufficiency@10 whether the set jointly supports the answer, and nDCG@10 rank quality [52]. Table II shows that QGMEM combines 89.08% Recall@10 with the best Precision@10 (20.92%), Sufficiency@10 (97.11%), and nDCG@10 (82.50%). Precision and nDCG improve by 3.00 and 14.34 points, indicating that answer-supporting memories are both covered and concentrated at high ranks; the sufficiency score confirms coverage of combined multi-evidence requirements.

5) *Adaptability Across LLM Backbones*: We compare Qwen3-8B, Llama3.1-8B-Instruct [18], and GPT-4o-mini [19]

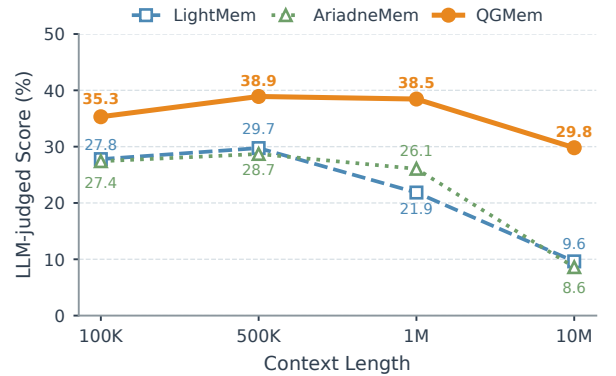


Fig. 4. Scalability on BEAM with Qwen3-8B. All methods use the same questions and histories within each scale, while the evaluation sets differ across scales.

on four benchmarks. Figure 5 shows the highest average for every backbone: 64.58%, 56.00%, and 70.11%, exceeding the strongest corresponding baselines by 13.57, 5.31, and 13.80 points.

QGMEM leads on LoCoMo, LongMemEval-S, and BEAM-100K for all three backbones, and reaches 78.10%/81.25% on Fact-SH with Qwen3-8B/GPT-4o-mini. GPT-4o-mini receives a textual serialization because its embedding space is inaccessible; its gains show that event-indexed memory and query-aware organization transfer across graph-token and text-only interfaces.

6) *Efficiency Analysis*: Table III reports online LLM and retrieved-context tokens, end-to-end online latency, and amor-

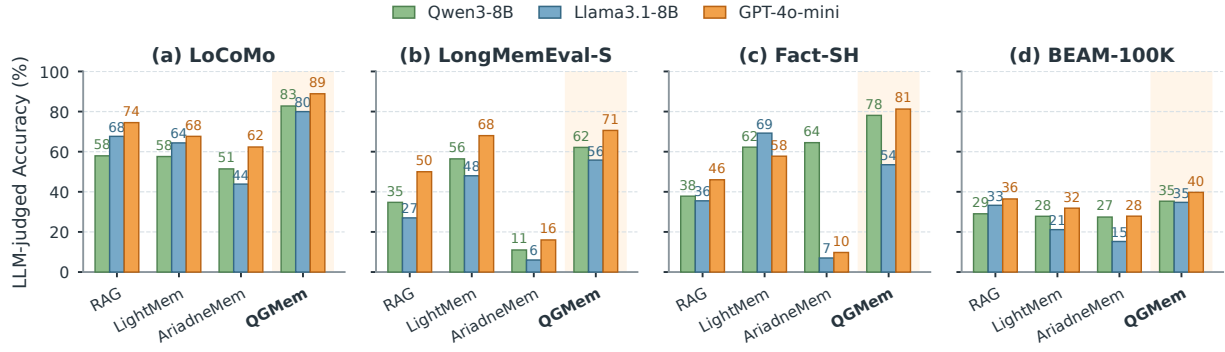


Fig. 5. Adaptability across LLM backbones on four benchmarks with Qwen3-8B, Llama3.1-8B-Instruct, and GPT-4o-mini.

TABLE II

THE EVIDENCE RETRIEVAL PERFORMANCE OF QGMEM COMPARED WITH REPRESENTATIVE BASELINES. HIGHER VALUES INDICATE BETTER PERFORMANCE FOR ALL METRICS.

Method	R@10	P@10	Suff.@10	nDCG@10
RAG	90.34	17.92	96.71	68.16
Mem0	82.77	16.42	86.71	48.13
LightMem	80.67	11.04	89.60	43.46
EMem-G	78.57	5.09	80.35	26.79
LiCoMemory	81.51	12.54	84.97	47.46
MR-Agent	55.88	3.99	45.09	28.73
AriadneMem	55.46	8.55	56.07	28.56
QGMEM (ours)	89.08	20.92	97.11	82.50

TABLE III

EFFICIENCY ON LoCoMo WITH QWEN3-8B. ONLINE LLM TOKENS INCLUDE ALL QUERY-TIME CALLS; OFFLINE LATENCY IS AMORTIZED PER QA.

Method	Online		Ret. ctx.	Offline
	tok./QA	Lat. (s/QA)	tok./QA	amort. s/QA
RAG	3430.01	3.84	3194.50	0.02
Mem0	879.68	3.49	656.12	5.07
LightMem	819.29	2.99	620.79	1.48
LiCoMemory	616.48	5.13	391.95	0.57
EMem-G	1505.78	7.18	563.35	0.24
QGMEM (ours)	1208.79	5.94	598.19	1.09

tized offline construction latency on LoCoMo. QGMEM uses 598.19 retrieved-context and 1208.79 total online tokens per QA, completes the full retrieval-to-generation pipeline in 5.94 seconds, and improves the LoCoMo score over EMem-G by 20.29 points. The 1.09-second offline value amortizes one-time construction of atomic units, traces, and indexes across all questions. Because the local graph is induced from fixed-budget working memory, longer histories enlarge the retrieval store without increasing the maximum graph supplied to the encoder or generator.

C. Ablation and Analysis

1) *Component ablation*: Table IV reports five variants; Macro Avg. is the unweighted mean across six benchmarks. For memory construction, excluding atomic memory units produces the largest decrease in Macro Avg. at 7.43 points. The decreases are 14.12 points on LongMemEval-S, 8.03/7.75 on BEAM-100K/500K, and 5.42 on Fact-MH. Event-level retrieval granularity therefore affects both long-history retrieval and multi-fact composition. Dynamic memory traces contribute

2.08 points to Macro Avg., with larger effects on Fact-MH (2.89 points) and BEAM-500K (4.23 points), two settings that place greater demands on tracking distant state changes.

For memory retrieval, excluding the reranker decreases Macro Avg. by 5.23 points, the second-largest change in the table. The corresponding decreases are 8.95/9.58 points on LoCoMo/LongMemEval-S and 2.53/5.53 points on BEAM-100K/500K. Candidate fusion therefore requires query-specific prioritization before answer generation. Excluding hybrid retrieval has a smaller overall effect of 1.44 points, but decreases Fact-SH and BEAM-100K by 2.78 and 2.30 points, respectively. The three retrieval channels thus improve candidate coverage before reranking.

For graph-augmented generation, excluding the graph token decreases Macro Avg. by 3.95 points. The decreases on Fact-SH, LongMemEval-S, Fact-MH, and BEAM-500K are 7.81, 7.06, 3.40, and 4.91 points, respectively. The reductions across these benchmarks suggest that graph topology complements the temporal and state-validity information carried by dynamic traces and serialized working memory.

2) *Qualitative analysis*: Figure 6 illustrates three failures: Mem0 retrieves “a partner to dance with” only at Rank 9; LightMem retrieves it at Rank 8 but answers “a marathon”; and EMem-G selects a nearby motivation relation at Rank 4 and answers “a picture of progress.” These correspond to ranking, evidence-utilization, and relation-selection errors.

QGMEM preserves the comparison as a traceable atomic unit, reranks it to Rank 1, and connects Jon and Gina’s shared challenges to the comparison in the local graph. Text and graph evidence therefore agree on “dancing with a partner,” illustrating how memory granularity, selective activation, and local relational organization address the three failure points.

3) *Hyperparameter sensitivity*: Figure 7 varies one parameter at a time. For the retrieval budgets, accuracy increases with working-memory size up to $K=10$ and remains below this maximum at every larger value evaluated. The initial increase is consistent with improved evidence coverage, while the subsequent reduction indicates that additional units can introduce irrelevant or redundant evidence. Candidate-pool size has a different trend. Accuracy is highest at $n=100$, and larger pools provide no consistent improvement, indicating that the reranker has sufficient candidate diversity at this value.

For memory construction, $w=6$ achieves the highest accuracy by providing sufficient context for local references while

TABLE IV

COMPONENT ABLATION OF QGMEM WITH QWEN3-8B ON EACH BENCHMARK. MACRO AVG. IS THE UNWEIGHTED MEAN ACROSS THE SIX BENCHMARKS, ASSIGNING EQUAL WEIGHT TO EACH BENCHMARK. HIGHER VALUES ARE BETTER; THE BEST AND SECOND-BEST RESULTS ARE **BOLDED** AND UNDERLINED RESPECTIVELY.

Setting	LoCoMo	LongMemEval-S	Fact-SH	Fact-MH	BEAM-100K	BEAM-500K	Macro Avg.
Full QGMEM	82.77	62.14	78.10	12.50	<u>35.31</u>	38.91	51.62
<i>Memory Construction Phase</i>							
w/o Dynamic Memory Traces	81.09	61.29	75.82	9.61	34.78	34.68	49.55 ^{↓2.08}
w/o Atomic Memory Units	79.84	48.02	71.75	7.08	27.28	31.16	44.19 ^{↓7.43}
<i>Memory Retrieval Phase</i>							
w/o Hybrid memory retrieval	<u>81.93</u>	<u>61.58</u>	75.32	<u>11.90</u>	33.01	<u>37.34</u>	<u>50.18</u> ^{↓1.44}
w/o Reranker	73.82	52.56	<u>76.91</u>	8.88	32.78	33.38	46.39 ^{↓5.23}
<i>Graph-Augmented Generation Phase</i>							
w/o Graph Token	81.80	55.08	70.29	9.10	35.75	34.00	47.67 ^{↓3.95}

Question: What did Jon and Gina compare their entrepreneurial journeys to?	
Mem0	Key Memory: rank 9 Jon and Gina find motivation in challenges, describing it as being like having a partner to dance with. Answer: Nothing is mentioned about comparing their entrepreneurial journeys. ✗
LightMem	Key Memory: rank 8 Jon mentions that having someone like Gina to face challenges with is like having a partner to dance with. Answer: A marathon. ✗
Emem-G	Key Memory: rank 4 Jon has been rehearsing and working on business plans ... but dancing has kept him going. Answer: A picture of progress. ✗
QGMEM	Key Memory: rank 1 Jon and Gina both face the same challenges ... it's like having a partner to dance with. Answer: Dancing with a partner. ✓

Fig. 6. A case study from LoCoMo. Mem0, LightMem, and EMem-G respectively illustrate ranking, evidence-utilization, and relation-selection failures, whereas QGMEM promotes the answer-bearing comparison to Rank 1 and produces the grounded answer.

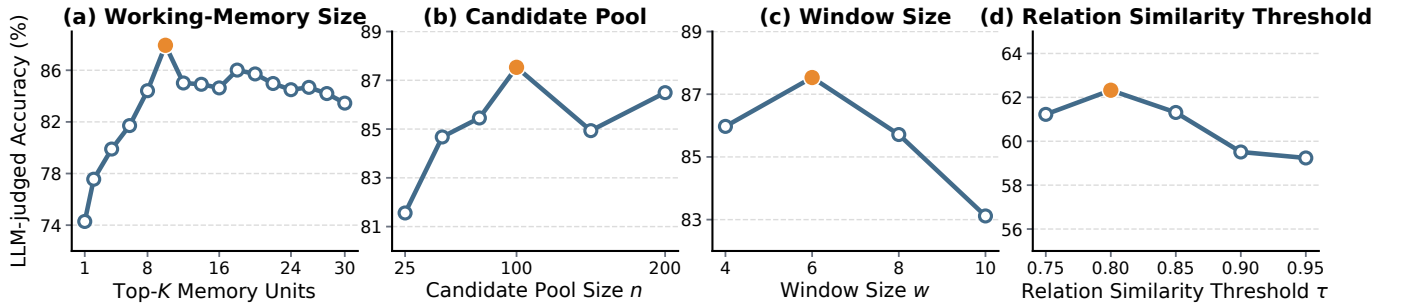


Fig. 7. Hyperparameter sensitivity of QGMEM with Qwen3-8B. Panels (a) through (d) vary working memory size, candidate-pool size, construction-window size, and relation similarity threshold respectively. Orange markers indicate the maximum on each curve.

retaining event-level separation. The neighboring settings remain competitive, although performance decreases when the window grows to ten turns. The macro-averaged result for relation consolidation is highest at $\tau=0.80$. Lower thresholds retain more weakly related facts, while higher thresholds remove useful cross-event connections. The observed values are consistent with a trade-off between relation coverage and relevance.

V. CONCLUSION

QGMEM is an event-centric conversational memory framework with query-aware graph augmentation. Its atomic units and dynamic traces preserve grounded evidence and evolving states, while query-aware activation organizes relevant units for local graph reasoning. Across six benchmarks and contexts up to 10M tokens, QGMEM improves retrieval, multi-hop reasoning, conflict resolution, and scalability across diverse backbones with compact contexts.

REFERENCES

- [1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. [Online]. Available: <https://aclanthology.org/2024.tacl-1.9/>
- [2] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, p. 186345, 2024.
- [3] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "Memorybank: Enhancing large language models with long-term memory," 2024. [Online]. Available: <https://arxiv.org/abs/2305.10250>
- [4] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "Memgpt: Towards llms as operating systems," 2023. [Online]. Available: <https://arxiv.org/abs/2310.08560>
- [5] J. Wang, S. Wang, Z. Xia, S. Hong, Y. Zhu, B. Liu, and C. Wu, "Fact: Examining the effectiveness of iterative context rewriting for multi-fact retrieval," 2024. [Online]. Available: <https://arxiv.org/abs/2410.21012>
- [6] Y. Wu, Y. Zhang, S. Liang, and Y. Liu, "Sgmern: Sentence graph memory for long-term conversational agents," 2025. [Online]. Available: <https://arxiv.org/abs/2509.21212>
- [7] S. Zhou and J. Han, "A simple yet strong baseline for long-term conversational memory of llm agents," 2025. [Online]. Available: <https://arxiv.org/abs/2511.17208>
- [8] C. Yang, C. Zhou, Y. Xiao, S. Dong, L. Zhuang, Y. Zhang, Z. Wang, Z. Hong, Z. Yuan, Z. Xiang, S. Chen, H. Zhou, Q. Zhang, N. Liu, J. Su, X. Wang, Y. Chang, and X. Huang, "Graph-based agent memory: Taxonomy, techniques, and applications," 2026. [Online]. Available: <https://arxiv.org/abs/2602.05665>
- [9] J. M. Zacks, N. K. Speer, K. M. Swallow, T. S. Braver, and J. R. Reynolds, "Event perception: A mind-brain perspective," *Psychological Bulletin*, vol. 133, no. 2, pp. 273–293, 2007.
- [10] G. A. Radvansky, "Across the event horizon," *Current Directions in Psychological Science*, vol. 21, no. 4, pp. 269–272, 2012.
- [11] G. A. Radvansky and J. M. Zacks, "Event boundaries in memory and cognition," *Current Opinion in Behavioral Sciences*, vol. 17, pp. 133–140, 2017.
- [12] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [13] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of llm agents," 2024. [Online]. Available: <https://arxiv.org/abs/2402.17753>
- [14] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "Longmemeval: Benchmarking chat assistants on long-term interactive memory," 2025. [Online]. Available: <https://arxiv.org/abs/2410.10813>
- [15] Y. Hu, Y. Wang, and J. McAuley, "Evaluating memory in llm agents via incremental multi-turn interactions," 2025. [Online]. Available: <https://arxiv.org/abs/2507.05257>
- [16] M. Tavakoli, A. Salemi, C. Ye, M. Abdalla, H. Zamani, and J. R. Mitchell, "Beyond a million tokens: Benchmarking and enhancing long-term memory in llms," in *International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=y59hf5lrMn>
- [17] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao *et al.*, "Qwen3 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [18] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur *et al.*, "The llama 3 herd of models," 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [19] OpenAI, "Gpt-4o mini: advancing cost-efficient intelligence," 2024, accessed: 2026-05-20. [Online]. Available: <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>
- [20] Z. Zhang, Q. Dai, X. Bo, C. Ma, R. Li, X. Chen, J. Zhu, Z. Dong, and J.-R. Wen, "A survey on the memory mechanism of large language model-based agents," *ACM Transactions on Information Systems*, vol. 43, no. 6, pp. 155:1–155:47, 2025.
- [21] Y. Hu, S. Liu, Y. Yue, G. Zhang, B. Liu, F. Zhu, J. Lin, H. Guo, S. Dou, Z. Xi, S. Jin, J. Tan, Y. Yin, J. Liu, Z. Zhang, Z. Sun, Y. Zhu, H. Sun, B. Peng, Z. Cheng, X. Fan, J. Guo, X. Yu, Z. Zhou, Z. Hu, J. Huo, J. Wang, Y. Niu, Y. Wang, Z. Yin, X. Hu, Y. Liao, Q. Li, K. Wang, W. Zhou, Y. Liu, D. Cheng, Q. Zhang, T. Gui, S. Pan, Y. Zhang, P. Torr, Z. Dou, J.-R. Wen, X. Huang, Y.-G. Jiang, and S. Yan, "Memory in the age of ai agents," 2026. [Online]. Available: <https://arxiv.org/abs/2512.13564>
- [22] W. Wang, L. Dong, H. Cheng, X. Liu, X. Yan, J. Gao, and F. Wei, "Augmenting language models with long-term memory," 2023. [Online]. Available: <https://arxiv.org/abs/2306.07174>
- [23] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready ai agents with scalable long-term memory," 2025. [Online]. Available: <https://arxiv.org/abs/2504.19413>
- [24] J. Fang, X. Deng, H. Xu, Z. Jiang, Y. Tang, Z. Xu, S. Deng, Y. Yao, M. Wang, S. Qiao, H. Chen, and N. Zhang, "Lightmem: Lightweight and efficient memory-augmented generation," 2026. [Online]. Available: <https://arxiv.org/abs/2510.18866>
- [25] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-mem: Agentic memory for llm agents," 2025. [Online]. Available: <https://arxiv.org/abs/2502.12110>
- [26] Y. Zhang, W. Yuan, and Z. Jiang, "Bridging intuitive associations and deliberate recall: Empowering LLM personal assistant with graph-structured long-term memory," in *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 2025, pp. 17 533–17 547. [Online]. Available: <https://aclanthology.org/2025.findings-acl.901/>
- [27] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," 2025. [Online]. Available: <https://arxiv.org/abs/2501.13956>
- [28] Z. Huang, Z. Tian, Q. Guo, F. Zhang, Y. Zhou, D. Jiang, Z. Xie, and X. Zhou, "Licomemory: Lightweight and cognitive agentic memory for efficient long-term reasoning," 2026. [Online]. Available: <https://arxiv.org/abs/2511.01448>
- [29] Z. Wu, H. Zhang, F. Lin, W. Xu, X. Xu, Y. Chen, H. P. Zou, S. Chen, W. Zhang, X. Liu, P. S. Yu, and H. Wang, "Gam: Hierarchical graph-based agentic memory for llm agents," 2026. [Online]. Available: <https://arxiv.org/abs/2604.12285>
- [30] J. Yue, C. Hu, J. Sheng, Z. Zhou, W. Zhang, T. Liu, L. Guo, and Y. Deng, "Hypermem: Hypergraph memory for long-term conversations," 2026. [Online]. Available: <https://arxiv.org/abs/2604.08256>
- [31] P. Banerjee, M. Moshtaghi, S. Subramanian, A. Misra, and A. Chadha, "Apex-mem: Agentic semi-structured memory with temporal reasoning for long-term conversational ai," 2026. [Online]. Available: <https://arxiv.org/abs/2604.14362>
- [32] H. P. Van, N. M. Hieu, K. P. T. Tuan, N. L. Hai, L. N. Van, N. T. N. Diep, and T. Le, "MemORAI: Memory organization and retrieval via adaptive graph intelligence for LLM conversational agents," in *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 2026, pp. 28 235–28 253. [Online]. Available: <https://aclanthology.org/2026.findings-acl.1408/>
- [33] B. Xu, Y. Chen, J. Fang, R. Zhong, Y. Yao, Y. Zhu, L. Du, and S. Deng, "StructMem: Structured memory for long-horizon behavior in LLMs," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, 2026, pp. 122–146. [Online]. Available: <https://aclanthology.org/2026.acl-short.12/>
- [34] S. Hu, Y. Wei, J. Ran, X. Han, Z. Yao, H. Wang, R. Chen, and L. Zou, "Does memory need graphs? a unified framework and empirical analysis for long-term dialog memory," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2026, pp. 26 758–26 782. [Online]. Available: <https://aclanthology.org/2026.acl-long.1232/>
- [35] S. E. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [36] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992. [Online]. Available: <https://aclanthology.org/D19-1410>
- [37] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781. [Online]. Available: <https://aclanthology.org/2020.emnlp-main.550>
- [38] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," *Transactions on Machine Learning Research*, 2022. [Online]. Available: <https://openreview.net/forum?id=jKN1pXi7b0>
- [39] W. Zhu, X. Chen, Z. Wang, J. Wang, X. Dong, M. Huang, R. Cai, H. Sang, H. Wang, P. Qiu, Y. Deng, P. Tiwari, B. H. Rappazzo, and

- Y. Wang, "Ariadnemem: Threading the maze of lifelong memory for llm agents," 2026. [Online]. Available: <https://arxiv.org/abs/2603.03290>
- [40] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," 2025. [Online]. Available: <https://arxiv.org/abs/2404.16130>
- [41] B. J. Gutierrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: <https://arxiv.org/abs/2405.14831>
- [42] J. Tang, Y. Yang, W. Wei, L. Shi, L. Su, S. Cheng, D. Yin, and C. Huang, "GraphGPT: Graph instruction tuning for large language models," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 491–500.
- [43] R. Chen, T. Zhao, A. K. Jaiswal, N. Shah, and Z. Wang, "LLaGA: Large language and graph assistant," in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235. PMLR, 2024, pp. 7809–7823. [Online]. Available: <https://proceedings.mlr.press/v235/chen24bh.html>
- [44] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, "G-retriever: Retrieval-augmented generation for textual graph understanding and question answering," 2024. [Online]. Available: <https://arxiv.org/abs/2402.07630>
- [45] P. Jiang, L. Cao, R. Zhu, M. Jiang, Y. Zhang, J. Shen, J. Sun, and J. Han, "Ras: Retrieval-and-structuring for knowledge-intensive llm generation," 2026. [Online]. Available: <https://arxiv.org/abs/2502.10996>
- [46] M. Huang, C. Bu, Y. He, X. Zhuo, and X. Wu, "Relink: Constructing query-driven evidence graph on-the-fly for graphrag," 2026, accepted by AAAI 2026. [Online]. Available: <https://arxiv.org/abs/2601.07192>
- [47] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, "Masked label prediction: Unified message passing model for semi-supervised classification," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, 2021, pp. 1548–1554.
- [48] H. Tan, Z. Zhang, C. Ma, X. Chen, Q. Dai, and Z. Dong, "MemBench: Towards more comprehensive evaluation on the memory of LLM-based agents," in *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 2025, pp. 19 336–19 352.
- [49] S. Ji, Y. Li, and B. Hooi, "Memory is reconstructed, not retrieved: Graph memory for llm agents," 2026, accepted at ICML 2026. [Online]. Available: <https://arxiv.org/abs/2606.06036>
- [50] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and chatbot arena," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2306.05685>
- [51] Y. Zha, Y. Yang, R. Li, and Z. Hu, "Alignscore: Evaluating factual consistency with a unified alignment function," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 11 328–11 348. [Online]. Available: <https://aclanthology.org/2023.acl-long.634>
- [52] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.
- [53] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-pack: Packed resources for general chinese embeddings," 2024. [Online]. Available: <https://arxiv.org/abs/2309.07597>
- [54] E. Pakhomov, E. Nijkamp, and C. Xiong, "Convomem benchmark: Why your first 150 conversations don't need rag," 2025. [Online]. Available: <https://arxiv.org/abs/2511.10523>
- [55] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang *et al.*, "Qwen3 embedding: Advancing text embedding and reranking through foundation models," 2025. [Online]. Available: <https://arxiv.org/abs/2506.05176>

APPENDIX

I. IMPLEMENTATION DETAILS

A. Implementation Settings

1) *Memory construction*: In the stage of memory construction, dialogue histories are first split at original session boundaries and then scanned within each session using fixed-size sliding windows. Unless otherwise specified, we use a window size of 6 turns and an overlap of 2 turns, resulting in a step size of 4 turns. Overlap turns are explicitly marked in the extraction prompt and are used only as background context for resolving pronouns and local references; the extractor is instructed not to create memory units whose evidence lies only in the overlap. The extractor may generate multiple memory units from the same window when different facts, preferences, events, or state changes are independently retrievable. BGE-base-en-v1.5 [53] is then used to embed memory-unit labels and graph records for retrieval and graph construction.

2) *Retrieval and generation*: For all LLM-driven steps, we set the decoding temperature to 0. During retrieval, the candidate pool size is set to 100 after RRF fusion over lexical, semantic, and event-attribute retrieval results, and reranking selects the best-performing budget of 10 memory units as working memory. For entity and location matching in the event-attribute retriever, dense semantic matching uses a similarity threshold of 0.60. When memory evolution is enabled, relation clustering uses a relation similarity threshold of 0.80 and a minimum relation-cluster size of 2.

B. Training Settings

Training data are constructed from ConvoMem [54], a conversational memory benchmark built from multi-turn dialogue histories. We draw examples from three deterministic-answer categories: user_evidence, assistant_facts_evidence, and changing_evidence, which respectively test user fact recall, assistant fact recall, and updated information tracking. These categories cover memory capabilities aligned with LoCoMo and LongMemEval-S, such as user/assistant information recall, temporal grounding, and knowledge update. We sample the three categories using a fixed 5:3:4 ratio, yielding 1,200 cases and 2,138 QA pairs.

For each case, we run the full QGMem pipeline and record the trajectory information, including the activated memory units, their serialized textual content, the local memory graph, the generated answer, and the reasoning path. We retain only cases with correct final answers, resulting in 1,634 judge-correct graph-grounded QA examples. We split the retained examples into train/validation/test partitions, and checkpoint selection is based only on the validation split. No LoCoMo or LongMemEval-S examples are used in this part. The retained ConvoMem examples and their trajectory information are used as supervision for training the memory graph token. Node and edge labels are represented with 768-dimensional BGE-base-en-v1.5 embeddings. The target is a JSON sequence containing answer and reasoning_path fields.

TABLE V
TRAINING HYPERPARAMETERS FOR THE MEMORY GRAPH TOKEN.

Hyperparameter	Value
Backbone LLM	Qwen3-8B, Llama3.1-8B-Instruct
LoRA	Disabled
Trainable modules	Graph encoder, projector
Graph encoder	4 TransformerConv layers
Hidden size	768
Attention heads	4
Pooling	Mean pooling
Projector	768 $\rightarrow$ 2048 $\rightarrow$ 4096
Batch size	4
Gradient accumulation	4
Learning rate	1×10^{-4}
Weight decay	0.05
Training epochs	10
Warmup epochs	1

C. Evidence Retrieval Quality Evaluation Protocol

Since different memory systems use different memory granularities, we evaluate retrieval quality at the gold-evidence level rather than by exact text overlap. We use the full LoCoMo evaluation set under the Qwen3-8B setting and compare QGMEM with the complete LoCoMo baseline suite: RAG, Mem0 [23], LightMem [24], EMem-G [7], LiCoMemory [28], MR-Agent [49], and AriadneMem [39]. All systems use the full evaluation set and the same retrieval cutoff $\ell = 10$. For each question, let $\mathcal{E}^*$ denote the gold-evidence set and $\widehat{\mathcal{M}}_\ell$ denote the top- ℓ retrieved memory units. A GPT-4o-mini judge identifies the covered subset $\text{Covered}(\widehat{\mathcal{M}}_\ell, \mathcal{E}^*) \subseteq \mathcal{E}^*$, the relevant subset $\text{Relevant}(\widehat{\mathcal{M}}_\ell, \mathcal{E}^*) \subseteq \widehat{\mathcal{M}}_\ell$, and whether the retrieved set contains sufficient answer-supporting evidence. We compute:

$$\text{Recall@}\ell = \frac{|\text{Covered}(\widehat{\mathcal{M}}_\ell, \mathcal{E}^*)|}{|\mathcal{E}^*|}, \quad (9)$$

$$\text{Precision@}\ell = \frac{|\text{Relevant}(\widehat{\mathcal{M}}_\ell, \mathcal{E}^*)|}{\ell}, \quad (10)$$

and $\text{Sufficiency@}\ell = 1$ if the retrieved set as a whole contains sufficient answer-supporting evidence to answer the question, and 0 otherwise. Unlike a conventional hit indicator, which is satisfied by any one relevant retrieval, sufficiency requires the combined retrieved evidence to cover the information needed for the answer. To measure ranking quality, let $y_i \in \{0, 1\}$ indicate whether the memory unit at rank i is relevant, and let $\text{IDCG@}\ell$ denote the DCG of the ideal ordering for the same number of relevant retrieved memories. We compute

$$\text{nDCG@}\ell = \frac{1}{\text{IDCG@}\ell} \sum_{i=1}^{\ell} \frac{y_i}{\log_2(i+1)}, \quad (11)$$

Recall is micro-averaged over gold items. Precision@10 uses the fixed denominator $10N$ across N questions, treating unfilled ranks as non-relevant, while Sufficiency@10 and nDCG@10 are averaged over questions. All scores in the evidence-retrieval table of the main paper are reported as percentages. Gold evidence and judge labels are used only for evaluation and are unavailable to all retrieval systems.

TABLE VI
ABLATION OF RETRIEVAL STEPS ON LoCoMo WITH QWEN3-8B.

Setting	Avg. Score
Full activation pipeline	82.77
w/o Hybrid memory retrieval	81.93
w/o Event-attribute retrieval	71.67
w/o reranking	73.82

D. Experiment Details

1) *Experimental Settings*: To ensure comparability, all systems use the same answer format, LLM-as-judge setting, and number of retrieved entries under each benchmark. Semantic retrieval uses BGE-base-en-v1.5 [53], and reranking uses Qwen3-Reranker-8B [55].

E. Additional Ablation Studies

1) *Retrieval and Selection Ablation*: Table VI ablates the retrieval and selection choices in working memory activation on LoCoMo with Qwen3-8B. Removing reranking leads to the largest drop, showing that semantic refinement after hybrid memory retrieval is important for selecting memory units containing answer-supporting evidence. Removing event-attribute retrieval also degrades performance, confirming that explicit entity, location, and time constraints help rank candidates matching the event attributes ahead of topically related but mismatched ones. Removing hybrid memory retrieval has a smaller but consistent effect, suggesting that the initial candidate pool improves recall before reranking.

II. PROMPTS

F. LLM-AS-JUDGE

1) LoCoMo Judge Prompt:

LoCoMo Judge Prompt

You are an evaluator for long-term conversational question answering. Given a question, a reference answer, and a model prediction, determine whether the prediction correctly answers the question. A prediction should be judged as correct if it is semantically equivalent to the reference answer, even if the wording is different. Minor paraphrases are acceptable. A prediction should be judged as incorrect if it contradicts the reference answer, misses a key entity, event, location, time, or relation, gives an answer that is too vague, or contains unsupported information.

Question: {question}
Reference answer: {reference_answer}
Model prediction: {model_prediction}

Return only a JSON object with two fields: {"judgment": "correct" or "incorrect", "reason": "brief explanation"}

2) LongMemEval-S Judge Prompt:

Single-session-user/assistant Multi-session

I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response is equivalent to the correct answer or contains all the intermediate steps to get the correct answer, you should also answer yes. If the response only contains a subset of the information required by the answer, answer no.

Question: {question}
Correct Answer: {answer}
Model Response: {response}

Is the model response correct? Answer yes or no only.

Temporal Reasoning Tasks

I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response is equivalent to the correct answer or contains all the intermediate steps to get the correct answer, you should also answer yes. If the response only contains a subset of the information required by the answer, answer no. In addition, do not penalize off-by-one errors for the number of days. If the question asks for the number of days/weeks/months, etc., and the model makes off-by-one errors (e.g., predicting 19 days when the answer is 18), the model's response is still correct.

Question: {question}

Correct Answer: {answer}

Model Response: {response}

Is the model response correct? Answer yes or no only.

Knowledge Update Tasks

I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response contains some previous information along with an updated answer, the response should be considered as correct as long as the updated answer is the required answer.

Question: {question}

Correct Answer: {answer}

Model Response: {response}

Is the model response correct? Answer yes or no only.

Single-session Preference Tasks

I will give you a question, a rubric for desired personalized response, and a response from a model. Please answer yes if the response satisfies the desired response. Otherwise, answer no. The model does not need to reflect all the points in the rubric. The response is correct as long as it recalls and utilizes the user's personal information correctly.

Question: {question}

Rubric: {answer}

Model Response: {response}

Is the model response correct? Answer yes or no only.

Abstention Tasks

I will give you an unanswerable question, an explanation, and a response from a model. Please answer yes if the model correctly identifies the question as unanswerable. The model could say that the information is incomplete, or some other information is given but the asked information is not.

Question: {question}

Explanation: {answer}

Model Response: {response}

Does the model correctly identify the question as unanswerable? Answer yes or no only.

1) *Prompts for QGMem*: The following templates follow the current implementation. Atomic memory construction has two LLM stages. The first stage segments an annotated sliding window and returns one source-grounded claim and one canonical triple per memory unit. The second stage receives those memory units in a batch and returns retrieval indexes keyed by the unchanged `memory_unit_id`. Event and activity terms are included in entities; the current implementation does not use a separate event-index field. The final-answer variable `{serialized_graph}` contains the complete textual generation context: evidence clusters with labeled memory-unit text and trajectories, followed by labeled structured facts. For open-weight backbones, the learned graph token is injected at the embedding level and therefore has no textual prompt placeholder.

Atomic Memory Unit Extraction Prompt

[System]

You are a long-conversation memory-unit extractor.

Input

- MEMORY_CONTENT contains a fixed-size sliding window of dialogue lines in this form:
NEW_IN_THIS_WINDOW [source_id] time: <timestamp> speaker: <message>
OVERLAP_FROM_PREVIOUS_WINDOW [source_id] time: <timestamp> speaker: <message>
- OVERLAP_FROM_PREVIOUS_WINDOW lines were already included in the previous sliding window. Do not extract memory units from these turns; they serve only as background information for resolving references and ambiguity for NEW_IN_THIS_WINDOW turns.
- NEW_IN_THIS_WINDOW lines are new turns in the current sliding window.

Task

- Segment the current sliding window into atomic, self-contained memory units. Each memory unit should express exactly one independently retrievable claim: one event, state, plan, preference, or other information.
- If the same turn supports multiple independent claims, output multiple memory units with the same start_turn/end_turn when needed. A unit may cover multiple turns only when those turns are needed to make the single claim complete.

Evidence bound

- Each memory unit MUST include start_turn and end_turn chosen only from source_ids shown in MEMORY_CONTENT.

Memory-unit fact_text

- fact_text is compact answer evidence. It must be specific enough to answer a later question without reading MEMORY_CONTENT.
- Resolve pronouns and speaker-relative references into the most specific supported entity, role, object, or group. Prefer real names when available; use 'the user' or 'the assistant' only when no more specific identity is given.

Triple extraction

- Output one 'triple' for the core claim of each memory unit. It must be directly supported by the unit and include subject, predicate, and object.

Output format

Return valid JSON only. If there is no retrievable factual content with non-overlap evidence, return {"memory_units": []}.

```
{
  "memory_units": [
    {
      "start_turn": "source id from MEMORY_CONTENT",
      "end_turn": "source id from MEMORY_CONTENT",
      "fact_text": "standalone reusable memory-unit text",
      "triple": {
        "subject": "...",
        "predicate": "...",
        "object": "..."
      }
    }
  ]
}
```

[User template]

Construct reusable memory units from this fixed sliding-window dialogue segment.

Session id: {session_id}
 Turn range: {start_turn}–{end_turn}
 Utterance time: {utterance_time}

MEMORY_CONTENT:
 {annotated_memory_content}

Output schema:

```
{
  "memory_units": [
    {
      "start_turn": "...",
      "end_turn": "...",
      "fact_text": "...",
      "triple": {
        "subject": "...",
        "predicate": "...",
        "object": "..."
      }
    }
  ]
}
```

Memory-Unit Retrieval Index Prompt

[System]

You extract retrieval indexes for memory-unit candidates from one dialogue segment.

Input contains a list of memory units. Each unit has memory_unit_id, utterance_time, fact_text, and evidence_text.

For each memory unit:

- 1) Extract entities: explicit people, named entities, events, activities, or concrete object/topic keywords needed to retrieve this unit and distinguish it from other memories.
- 2) Extract locations: concrete places, venues, facilities, cities, countries, or regions where the fact happened. Do not put events or activities in locations unless the text names a physical venue.
- 3) Extract time: when the described fact happened. Each item is {"start": "...", "end": "...", "granularity": "day|week|month|year"}. Use [] for unknown or vague recency.

Time format:

- For an exact date, set start=end and use the matching granularity.
- For a relative but bounded time, such as "last weekend" or "two weeks before DATE", output a conservative date range rather than a guessed day.
- Distinguish utterance_time and fact time. Resolve relative expressions against the memory unit's own utterance_time, and do not copy utterance_time itself as the fact time.
- For vague times such as "recently", "lately", "a while ago", or "the other day", output [].

Return valid JSON only:

```
{
  "memory_units": [
    {
      "memory_unit_id": "...",
      "entities": ["..."],
      "locations": ["..."],
      "time": [
        {
          "start": "YYYY-MM-DD|YYYY-MM|YYYY",
          "end": "YYYY-MM-DD|YYYY-MM|YYYY",
          "granularity": "day|week|month|year"
        }
      ]
    }
  ]
}
```

[User template]

Extract retrieval indexes for these memory units from one dialogue segment.

Session id: {session_id}
 Turn range: {start_turn}–{end_turn}
 Segment utterance time: {utterance_time}

MEMORY_UNITS:
 [

```
{
  "memory_unit_id": "...",
  "utterance_time": "...",
  "fact_text": "...",
  "evidence_text": "..."
}
```

Dynamic Memory Trace Consolidation Prompt

[System]

You are responsible for turning candidate atomic memories into one concise memory trajectory. Several memory units may describe the same subject–relation, and later memories can update, refine, repeat, conflict with, or add to earlier memories. The input items are candidate members of one relation cluster, but some may be irrelevant or only loosely related.

Write one chronological trajectory supported by the memory units:

- keep only items that describe the same evolving subject–relation,
- order the relevant states by memory time,
- include both earlier history and the latest/current supported state.

Use memory order, not world knowledge:

- If time values are ‘serial–N’, larger N means newer memory.
- If a later memory contradicts an earlier one, describe the later memory as an update.

Return only valid JSON:

```
{
  "trajectory": "compact chronological trajectory including earlier
state(s) and the latest supported state"
}
```

[User template]

Write one memory trajectory for the following relation cluster.

RELATION:

{relation_text}

MEMORY_UNITS are sorted from oldest to newest by memory time.

Write the trajectory from earlier memory to the latest supported memory, including updates and non–conflicting additions.

{members_json}

Query Intent Parsing Prompt

[System]

You parse a question into retrieval constraints for a long–term memory store.

Background

- The memory store indexes each memory unit by ‘entities’, ‘locations’, and ‘time’.
- Your output is used to select candidate memories before answer generation.
- The extracted values are used for semantic retrieval, so they should be discriminative rather than general.

Task

- Read the question and extract retrieval constraints on three axes: ‘entities’, ‘locations’, and ‘time’.
- Keep each list concise and deduplicated.
- If an axis has no useful constraint, return an empty list for that axis.

Entity and Location

- Extract people, named objects, events, activities, roles, organizations, artifacts, and concrete topic words in ‘entities’.
- Extract physical or named places, such as Paris, Pacific Northwest, school, gym, museum, beach, or library, in ‘locations’.
- Do not put answer types in ‘locations’. For example, “countries”, “states”, “places”, or “where” describe what the answer should find.

- Do not put events or activities in ‘locations’; keep them in ‘entities’ when they constrain evidence.

Time

- ‘time.fact’ is for when the remembered fact happened, and ‘time.utterance’ is only for when the conversation happened, such as “in our chat on June 3” or “as mentioned last Tuesday”.
- Normalize clear dates as date–only values: ‘YYYY–MM–DD’ for a day or ‘YYYY–MM–DD..YYYY–MM–DD’ for an interval.

[User template]

Parse this query into retrieval–oriented intent.

Query:

{query}

Question time:

{question_time}

Use Question time only as the reference for explicit relative time expressions in the query. Do not output it as a time constraint by default.

Output schema:

```
{
  "entities": ["explicit person, named entity, or topic/object constraint"],
  "locations": ["concrete place or venue constraint stated in the query"],
  "time": {
    "fact": ["YYYY–MM–DD or YYYY–MM–DD..YYYY–MM–DD
for when the remembered fact happened"],
    "utterance": ["YYYY–MM–DD or
YYYY–MM–DD..YYYY–MM–DD only when the query
explicitly constrains when the conversation happened"]
  }
}
```

Final Answer Prompt

[System]

You answer questions using only the provided memory context.

Input

- QUESTION is the user’s question.
- QUESTION_TIME, when provided, is the time the question was asked. Use it only to resolve relative time expressions in the question.
- MEMORY_CONTEXT includes:
 - Evidence Clusters: retrieved memories grouped by memory–evolution relation. A cluster includes a ‘trajectory’ field describing earlier history and the latest state, together with labeled memory–unit text.
 - Facts: labeled structured subject–predicate–object facts.
- Memory–unit ‘utterance_time’ is when the dialogue happened; if memory–unit times use ‘serial–N’, larger N means newer evidence.

Task

- Produce the best supported answer to QUESTION.
- Ground the answer only in MEMORY_CONTEXT.
- When a trajectory’s latest state conflicts with older content or older Facts from the same relation, use the trajectory’s latest state as the latest memory–supported evidence.

Output Format

Think step by step internally, then return valid JSON only:

```
{
  "reasoning_path": "brief chain of reasoning citing labels (MU#/F#)
used",
  "answer": "final answer string only"
}
```

[User template]

Question:

{query}

Question time:
{question_time}

Graph:
{serialized_graph}

Return only valid JSON with keys reasoning_path and answer, in that order.